

TEAM COLLABORATION

Agentic AI Project Guide

For a Team of 8 · Full-Stack + AI-Assisted Development

ARCHITECTURE

WORKFLOW

CONTRACTS

AI TOOLING

2026 · Internal Reference Document

Table of Contents

PART 1 — AI-Assisted Team Setup

- Monorepo Structure
- CLAUDE.md — AI Context File
- Git Branching Strategy
- Module Ownership Matrix
- API Contract-First Approach
- AI Tool Recommendations
- Pre-Commit Quality Gates
- Communication Stack
- Quick Start Checklist

PART 2 — The Collaboration Ponder

- The Core Problem
- Splitting Work by System Layer
- The Biggest Collaboration Risk
- What Needs Coordination vs. What Doesn't
- Practical Team Rhythm
- The AI Context Document
- Integration Testing — The Overlooked Step

PART 3 — Deep Dive — Complex Agentic System

- Agent Orchestration Layer
- Tool Layer
- Memory & Context Layer
- DB & Storage Layer
- Frontend / Interface Layer
- Infra & DevOps Layer
- The Contract System
- Breaking Change Protocol
- AI Context Document (Full Detail)
- Weekly End-to-End Smoke Test
- The Mental Model Problem

PART ONE

Foundational tooling, structure, and conventions for 8 developers building with AI assistance across frontend, backend, and infra.

1. Monorepo Structure

With 8 people and AI assistants generating code simultaneously, having a single repository is non-negotiable. A monorepo ensures every AI instance operates on the same structural truth — same folder conventions, same imports, same tooling.

```
/project-root

/frontend ← Next.js / React UI

/backend ← FastAPI / Spring Boot

/agents ← Orchestration, state machines, planners

/tools ← Individual agent tool implementations

/memory ← Memory layer (vector store, retrieval logic)

/shared ← Types, constants, API contracts

/docs ← Architecture docs, API specs, decisions

/prompts ← Shared prompt templates

/contracts ← Tool, memory, DB, and API contracts

/infra ← Docker, Terraform, CI/CD

CLAUDE.md ← AI context file (every member feeds this to their AI)

.cursorrules ← Cursor-specific project rules
```

2. CLAUDE.md — The AI Context File

This is the single most impactful file in your repo. Every team member pastes this into their AI assistant at the start of each session. Without it, 8 AI instances will make 8 different architectural assumptions.

```
# Project: [Name]

## Stack

- Orchestration: LangGraph / CrewAI

- Memory: pgvector + Redis

- Backend: FastAPI + PostgreSQL

- Frontend: Next.js 14, Tailwind, shadcn/ui

- Infra: Docker, GitHub Actions, AWS
```

```
## Agent Tool Registry

- tool: web_search | input: {query: str} | output: {results: list}

- tool: db_query | input: {sql: str} | output: {rows: list}

- tool: send_email | input: {to, subject, body} | output: {status}


## Memory Schema

- Long-term: pgvector embeddings, metadata: {user_id, session_id, timestamp, type}

- Short-term: last N messages injected directly into context


## Naming Conventions

- Components: PascalCase | API routes: snake_case | Branches: feat/short-name


## Do NOT

- Modify tool output schema without updating /contracts/tools.md

- Write raw SQL outside the DB layer

- Hardcode prompts — all prompts live in /prompts/
```

Update this file every Friday as part of your team sync. Treat it like production code — version it, review changes, never let it go stale.

3. Git Branching Strategy

Use GitHub Flow — simple enough for a student team, robust enough for a complex system. The key rule: nobody pushes directly to main or dev.

```
main ← production-ready only (protected)

dev ← integration branch (protected)

feat/login ← individual feature branches

feat/agent-tool-x

feat/memory-schema-v2

fix/retrieval-bug

chore/update-deps
```

- **Branch protection:** Require at least 1 PR review before merging to dev.

- **AI-generated code** gets the same review scrutiny as human-written code — no exceptions.
- **Commit messages:** Use conventional commits (feat:, fix:, chore:, docs:, refactor:).

4. Module Ownership Matrix

Assign layer ownership, not just task assignment. Owners are the final decision-makers and primary reviewers for their layer.

Members	Owns	Responsibility
1	Agent Orchestration	State machine, planning logic, prompt architecture, retry/fallback
2	Tool Layer	Tool implementations, Tool Registry, input/output contracts
3	Memory & Context	Vector store, retrieval strategy, embedding model, memory schema
4	DB & Storage	Schema, migrations, Data Contract Document, Redis cache strategy
5–6	Backend / API	FastAPI routes, business logic, auth, service layer
7–8	Frontend / Interface	Next.js UI, streaming agent output, human-in-the-loop hooks
Any 1	Infra & DevOps	Docker, CI/CD, env management, observability (Langfuse)

5. API Contract-First Approach

The #1 source of team conflict: frontend expects an API response shape that doesn't match what the backend returns. Fix it before anyone codes.

```
# /contracts/api.yaml

POST /api/agent/run

request: { user_id: str, query: str, session_id: str }
response: { run_id: str, status: 'queued' | 'running' | 'done' }

GET /api/agent/stream/{run_id}

response: SSE stream of { step: str, tool: str, output: any, done: bool }

POST /api/auth/login

request: { email: str, password: str }
```

```
response: { token: str, user: { id, name, role } }
```

6. AI Tool Recommendations by Role

Role	Recommended Tool	Why
All members	Cursor + CLAUDE.md	Full repo context, not just current file
Complex logic / agents	Claude (claude.ai)	Best at multi-step reasoning and architecture
Backend API	Claude Code (terminal)	Full file access, can run and test
Frontend UI	v0.dev → customize	Rapid component scaffolding
Code review	CodeRabbit	AI PR reviewer, free for students
Observability	Langfuse	Agent trace visibility, prompt versioning

7. Pre-Commit Quality Gates

Stop broken AI-generated code from entering the repo automatically. Set these up in week 1 — they pay dividends all project long.

```
# .husky/pre-commit

# Frontend

npx eslint . --ext .ts,.tsx

npx prettier --check .

# Backend (Python)

black --check .

flake8 . --max-line-length=100

# Run fast unit tests

pytest tests/unit/ -q
```

- **GitHub Actions CI:** Tests run on every PR, full integration test on merge to dev.
- **AI writes tests too** — make it a team habit: no PR merges without test coverage.

8. Communication Stack

Purpose	Tool
Daily async updates / blockers	Discord or WhatsApp thread
Task tracking	GitHub Projects (free) or Notion
Design specs	Figma
Shared AI prompts	/prompts/ folder in repo
Architecture decisions	/docs/decisions/ (ADR format)
Breaking change proposals	GitHub Discussions or Notion page

9. Quick Start Checklist

- Create monorepo, invite all 8 members to GitHub with appropriate roles
- Write CLAUDE.md together in the first team meeting
- Define the API contract document before anyone writes backend code
- Set up branch protection on main and dev branches
- Assign layer ownership — one person clearly responsible per layer
- Install Husky + ESLint + Prettier (frontend) / Black + Flake8 (backend)
- Set up GitHub Actions CI pipeline (basic tests on every PR)
- Set up Docker Compose for one-command local full-stack run
- Create /contracts/ folder with tool, memory, DB, and API contract stubs
- Set up Langfuse for agent observability from Day 1
- Schedule twice-weekly team sync (30 min max)

PART TWO

How to think about the problem before picking tools. Strategic framing for complex, emergent agentic systems.

The Core Problem to Solve First

A complex agentic system is not like a standard web application. The codebase is deeply interconnected — an agent's memory layer talks to the DB, the DB schema affects tool outputs, tool outputs affect orchestration logic. If 8 people are building in silos, things break at the seams, not inside the modules.

“ The question is not just 'how do we not step on each other' — it is 'how do we make sure all 8 mental models of the system stay in sync.' ”

How to Think About Splitting the Work

Instead of splitting by feature or page, split by system layer. Each layer has one or two owners — not contributors, but decision-makers. This matters more in a complex system because one wrong schema change ripples everywhere.

```
[ Agent Orchestration ] ← Who owns the brain?  
  
[ Tool Layer ] ← Who builds what the agent can do?  
  
[ Memory & Context ] ← Who owns what the agent remembers?  
  
[ DB & Storage ] ← Who owns the data contracts?  
  
[ Frontend / Interface ] ← Who owns how humans interact with it?  
  
[ Infra & Deployment ] ← Who owns it running reliably?
```

The Biggest Collaboration Risk in Agentic Projects

The agent's behavior is emergent — it depends on prompts, tools, memory, and orchestration all working together. No single person sees the full picture. The team needs a shared place where:

- The **agent's full flow** is documented — what it can do, how it decides, what it remembers.
- Any **change to tools, memory schema, or prompts** is discussed before merging.
- The **DB contracts** are agreed upon before anyone builds on top of them.

Without this, two people can build technically correct modules that produce a broken agent when combined.

What Needs Coordination vs. What Doesn't

Needs TIGHT Coordination	Can Be Built INDEPENDENTLY
Agent state machine / orchestration logic	Individual tool implementations (once interface agreed)
Memory schema (storage format, retrieval)	Frontend UI components
Tool interface contracts (inputs/outputs)	Individual API endpoints
Prompt templates driving agent behavior	Infrastructure scripts
DB schema changes	Unit tests for isolated modules

The interface contracts are your collaboration glue. Define them early, protect them aggressively.

A Practical Rhythm for 8 People

- **Twice-weekly sync (30 min)** — only for cross-layer decisions and blockers, not status updates.
- **Async PR reviews** — 24-hour window for everyone to review in their own time.
- **One shared living doc** — the system truth document: agent flow, tool list, memory schema, DB contracts. Version it like code.
- **Breaking change protocol** — post a proposal first, give 24 hours for feedback, then proceed. No surprise changes to shared contracts.

Integration Testing — The Step Most Teams Skip

Most teams build all modules, then try to connect them in week 3 and discover nothing works together. For an agentic system, run a weekly end-to-end smoke test — even with stub tools and dummy memory — just to make sure the agent can complete one full flow. It forces integration issues to surface early when they are cheap to fix.

```
# Weekly Smoke Test Log Template

Week N Smoke Test

Flow tested: User asks agent to [describe task]

Result: PASS / FAIL

Steps taken: N

Issues found: [Tool X returned wrong format → issue #XX]

Contract drift detected: Yes / No

Action items: [list]
```

PART THREE

Detailed ownership responsibilities, the full contract system, breaking change protocol, and solving the mental model problem.

Layer 1 — Agent Orchestration

This is the hardest layer. The person here is the architect of the agent's brain. Assign your strongest systems thinker. A bad orchestration design early on is extremely painful to refactor because everything else depends on it.

Ownership Responsibilities:

- **State machine** — What states the agent can be in, what triggers transitions.
- **Planning logic** — How the agent decides which tool to call next.
- **Retry and fallback behavior** — What happens when a tool fails or returns garbage.
- **Prompt architecture** — System prompt structure, how context is injected.

```
# State Machine Example (LangGraph)

States: IDLE → PLANNING → TOOL_CALL → OBSERVING → RESPONDING → END

↓ (on tool failure)

RETRY (max 3) → FALLBACK → RESPONDING

Transitions triggered by:

- planner output (next tool or final answer)
- tool result (success / error / empty)
- human interrupt signal
```

Layer 2 — Tool Layer

Every capability the agent has is a tool. Tool owners build individual tools and maintain the central Tool Registry. Nobody ships a new tool without adding it to the registry.

Each Tool Must Specify:

```
Name: The identifier the agent uses to call this tool

Description: Natural language – how the agent knows WHEN to use it

Input schema: Exact types and required/optional fields

Output schema: Exact structure the orchestrator expects back

Error cases: What it returns on failure (never throw raw exceptions)

Owner: Which team member is responsible for this tool
```

Status: `stable` | `in-progress` | `deprecated`

Tool Registry (sample):

Tool Name	Input	Output	Status
web_search	{query: str}	{results: list[{title, url, snippet}]}	stable
db_query	{sql: str, params: list}	{rows: list, count: int}	stable
send_notification	{user_id, message, channel}	{delivered: bool}	in-progress
summarize_doc	{doc_id: str, max_tokens: int}	{summary: str}	stable

Layer 3 — Memory & Context

The most underestimated layer. Memory in a production agentic system is not just 'save to database' — it involves multiple memory types that interact in subtle ways with the agent's behavior.

Memory Type	Description	Implementation
Short-term	Current context window contents	Last N messages injected directly
Long-term	Persisted knowledge across sessions	Vector embeddings (pgvector / Qdrant)
Episodic	Logs of past agent runs	Structured DB + summarization
Working memory	Intermediate state during multi-step task	In-memory dict / Redis

Key Design Decisions (decide before anyone builds on top):

- **What gets stored and when** — not everything should be persisted.
- **Embedding strategy** — which model (OpenAI, Cohere, local), what chunking approach.
- **Retrieval strategy** — similarity threshold, top-k, reranking, MMR.
- **Context injection** — exactly how retrieved memory enters the agent's prompt.

“ A bad memory schema early is catastrophic. If you store the wrong things, retrieval is noisy. If you store in the wrong format, you cannot query efficiently. Design this carefully — it cannot be easily migrated once the agent is running. ”

Layer 4 — DB & Storage

This person owns all persistent storage and is responsible for the Data Contract Document — the source of truth for every table, collection, and field across the system.

- **Relational DB** (PostgreSQL) — users, sessions, agent runs, tool call logs.
- **Vector DB** (pgvector or Qdrant) — embedding collections, metadata schema, index config.
- **Cache** (Redis) — what's cached, TTL strategy, cache invalidation rules.
- **Migration files** — every schema change goes through Alembic migrations. No manual alterations.

Data Contract Document — Required Fields per Table:

Table	Key Fields	Purpose
agent_runs	run_id, user_id, session_id, status, started_at, completed_at	Track every agent execution
tool_calls	call_id, run_id, tool_name, input_json, output_json, duration_ms	Full tool call audit log
memory_store	mem_id, user_id, content, embedding, metadata_json, created_at	Long-term vector memory
sessions	session_id, user_id, context_json, last_active	Active session state

Layer 5 — Frontend / Interface

For an agentic system, the frontend is not just UI — it is how humans observe and interact with an autonomous process. That is a fundamentally different design problem from a standard CRUD app.

- **Real-time streaming** — SSE or WebSockets to show agent output as it happens.
- **Agent state visibility** — what step is it on, which tool is it calling, why.
- **Human-in-the-loop hooks** — UI pause points where the agent waits for human confirmation.
- **Run history** — browsable logs of past agent sessions with full trace and tool call details.
- **Error and fallback states** — graceful UI for when the agent fails or times out.

Layer 6 — Infra & DevOps

Set up Docker Compose in week 1, before anyone writes real feature code. If developers cannot run the full system locally, integration bugs hide until too late.

```
# docker-compose.yml (minimum viable setup)

services:

  postgres: image: pgvector/pgvector:pg16
```

```
redis: image: redis:7-alpine

backend: build: ./backend ports: 8000:8000

frontend: build: ./frontend ports: 3000:3000

langfuse: image: langfuse/langfuse (agent observability)

# One command to run everything:

$ docker-compose up
```


The Contract System

Contracts are the API between people, not just between services. In a complex agentic system there are four types of contracts. Each lives in the `/contracts/` folder, versioned like code.

Contract Type	What It Defines	Who Maintains It
Tool Contracts	Input/output schema for every agent tool	Tool Layer owner
Memory Contracts	What gets stored, format, how retrieved	Memory Layer owner
DB Contracts	Schema, field types, relationships, indexes	DB Layer owner
API Contracts	Endpoints, request/response shapes	Backend + Frontend jointly

When anyone wants to change a contract, they open a proposal in `/docs/proposals/`, the affected layer owners review it, and it gets merged only after sign-off. This sounds bureaucratic, but for 8 people on a complex system it saves hours of debugging mystery failures caused by silent contract drift.

The Breaking Change Protocol

This is the specific process for when a change affects more than one layer. Any change that touches a shared contract must go through this flow — no exceptions.

1 Author writes a proposal	What is changing · Which layers are affected · Why it is necessary · Migration plan for existing data/code
2 Post to team channel	Tag the owners of all affected layers. Include a link to the proposal file in <code>/docs/proposals/</code> .
3 24-hour review window	Affected layer owners flag concerns or approve. Silence after 24h is treated as implicit approval.
4 Resolve and merge	No blockers → merge. Blockers → 15-minute call to resolve, then merge. Document the outcome.

Examples that trigger this protocol:

- Adding a field to an agent tool's output schema.
- Changing how memory is retrieved (affects what the orchestrator sees).
- Renaming a DB table or column.
- Changing the agent's planning prompt structure.
- Modifying the session context format.

The AI Context Document — Full Structure

This is not documentation for humans — it is runtime context for the AI assistant. Every team member feeds this to their AI at the start of every session. Update it every Friday. One person owns the update, everyone reviews the diff.

```
# =====  
  
# CLAUDE.md — AI CONTEXT DOCUMENT  
  
# =====  
  
## System Overview  
  
This is an agentic AI platform that [one clear sentence purpose].  
  
The agent can: [list of capabilities].  
  
## Tech Stack  
  
- Orchestration: LangGraph (graph-based, stateful multi-step agent)  
- Memory: pgvector (long-term) + Redis (working memory)  
- Backend: FastAPI + PostgreSQL + Redis  
- Frontend: Next.js 14 + Tailwind + shadcn/ui  
- Infra: Docker Compose, GitHub Actions, AWS ECS  
- Observability: Langfuse (traces, prompt versions, evaluations)  
  
## Current Tool Registry  
  
- web_search | input: {query: str} | output: {results: list}  
- db_query | input: {sql: str} | output: {rows: list}  
- send_email | input: {to, subject, body} | output: {status: str}  
  
# (full list in /contracts/tools.md)  
  
## Memory Schema  
  
- Long-term: embeddings stored in pgvector, metadata: {user_id, session_id,  
timestamp, type, source}  
  
- Short-term: last 10 messages injected directly into system prompt
```

```
- Retrieval: top-5 by cosine similarity, threshold 0.75, then reranked

## Folder Conventions

- Agent logic: /agents/

- Tool impls: /tools/

- Memory logic: /memory/

- Prompt files: /prompts/

- DB models: /backend/models/

- Contracts: /contracts/

## Naming Conventions

- Components: PascalCase | API routes: snake_case | Branches: feat/short-name

- Tool names: snake_case verbs (web_search, not webSearch or WebSearch)

## Hard Rules

- Never modify tool output schema without updating /contracts/tools.md

- Never write raw SQL outside /backend/repositories/

- All prompts live in /prompts/ – never hardcode in agent logic

- All DB changes need a migration file in /backend/migrations/
```

The Mental Model Problem — And How to Solve It

The deepest challenge with 8 people on an agentic system is that nobody holds the full picture in their head. The orchestration owner does not know the exact DB schema. The DB owner does not know how the agent uses memory. The frontend owner does not know the agent's planning logic. This is normal — and dangerous.

The Fix: A Living System Map

Maintain a single diagram updated monthly that every member of the team has seen and understands. It does not need to be fancy. A clean diagram updated regularly is worth more than a detailed one that is always out of date.

The system map must show:

- Every layer and who owns it.

- How data flows between layers (what calls what, what format).
- Where the agent makes decisions (planning, tool selection, fallback).
- Where human input enters the loop (HITL checkpoints).
- Where memory is read and written.

“ On a project this complex, your real product is the system’s coherence across 8 people’s work — not just the features each person ships individually. Protect the contracts, keep the mental models aligned, and the code almost takes care of itself. ”

Summary — Key Principles at a Glance

Principle	Why It Matters
Own layers, not tasks	Prevents diffused responsibility on interconnected systems
Define contracts before coding	Eliminates the #1 integration failure mode
Breaking change protocol	Prevents silent contract drift that compounds into debugging hell
CLAUDE.md as living document	Keeps 8 AI instances aligned on the same architecture
Weekly end-to-end smoke test	Surfaces integration failures when they are still cheap to fix
System map for mental model alignment	Prevents people from building on wrong assumptions
One-command Docker local setup	Eliminates 'works on my machine' and unblocks everyone
Langfuse from Day 1	Makes agent behavior visible and debuggable — not a black box

END OF DOCUMENT

Version 1.0 · 2026 · Internal Reference